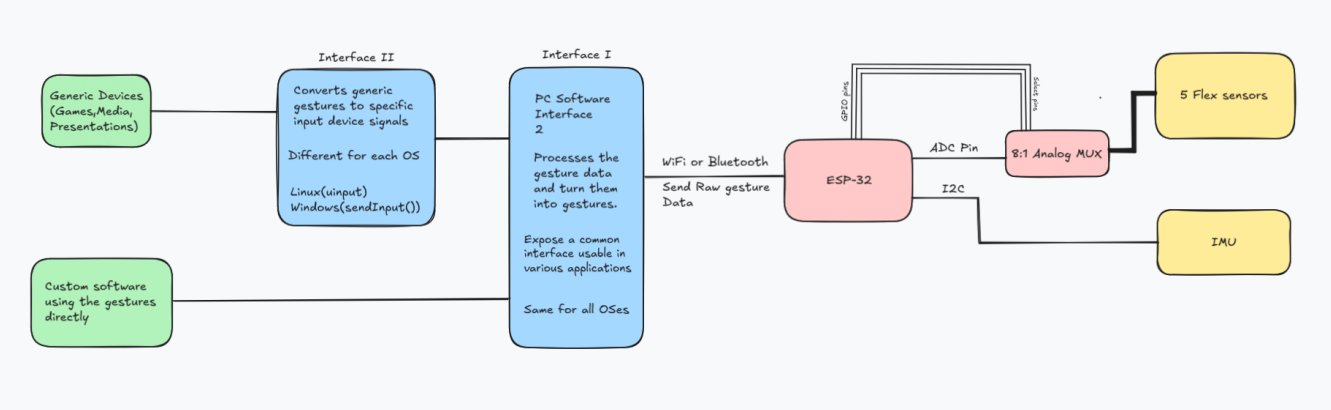
Data Glove Gesture Recognition System

This document explains the architecture of a wearable gesture-recognition glove. The system reads finger movements and hand orientation from hardware sensors, processes that data on a microcontroller, and transmits it wirelessly to a PC. On the PC, software recognises the gestures and either turns them into standard keyboard/mouse-style input for existing applications or hands the raw gesture data to custom software. The design is deliberately split into layers so each part can change without breaking the rest, and so the same core works across Linux and Windows.



The system flows left to right: applications consume input on the left, PC software processes gestures in the middle, and the glove hardware senses and transmits on the right. The sections below walk through each layer in that same direction.

1. Sensor Layer — What Your Hand Is Doing

The glove carries two kinds of sensor, each answering a different question.

Flex Sensors (5x)

Five bend sensors sit along the fingers. As a finger curls, the sensor's electrical resistance changes, and that change is read as an analog voltage. Together the five sensors tell the system how much each finger is bent — the core information needed to tell one hand gesture from another. They are simple, cheap, and give a continuous value (not just open/closed).

IMU (Inertial Measurement Unit)

The IMU is a 9-axis sensor combining an accelerometer, a gyroscope, and a magnetometer. It answers the question the flex sensors cannot: what is the hand doing as a whole — its tilt, rotation, and motion through space? Flex sensors capture finger pose; the IMU captures hand orientation and movement, so gestures like *tilt to scroll* or *rotate the wrist* become possible.

2. Data Acquisition — Getting Sensor Signals Into One Chip

8:1 Analog Multiplexer (MUX)

The ESP-32 has a limited number of analog input pins, but we have five flex sensors. Instead of dedicating five pins, an 8:1 multiplexer acts as a rotary switch: the ESP-32 sets three select pins (GPIO) to choose which of up to eight input channels is connected to a single ADC pin at any moment. The microcontroller then cycles through the channels rapidly, reading each flex sensor one at a time through that one ADC pin.

ESP-32 Microcontroller

The ESP-32 is the brain of the glove. It reads the flex sensors through the MUX via its ADC pin, reads the IMU directly over I2C (a two-wire serial bus), and assembles everything into a stream of raw sensor data. Because it has built-in WiFi and Bluetooth, it then transmits that stream wirelessly to the PC. Its interfaces each have a distinct job:

Interface	Connects To	Purpose
ADC pin	8:1 MUX output	Reads the currently selected flex sensor's analog value
I2C	IMU	Reads orientation and motion data from the 9-axis IMU
GPIO (select pins)	MUX select lines	Chooses which of the 8 MUX channels is active
WiFi / Bluetooth	PC	Sends the assembled raw data wirelessly

A key design point: the ESP-32 sends *raw* data — unfiltered sensor readings — not recognised gestures. All recognition happens on the PC. This keeps the glove simple and low-power, and lets the recognition logic be improved without touching the firmware.

3. PC Software — Turning Raw Data Into Useful Input

The PC software is split into two interfaces, so that gesture recognition is cleanly separated from operating-system specifics.

Interface I — Recognition & Common API

This layer receives the raw data stream from the ESP-32 over WiFi or Bluetooth, processes it, and recognises gestures (for example, *index finger bent while thumb extended* maps to a specific action). It then exposes a single, clean interface for the rest of the system to consume. Critically, this layer is **identical on every operating system** — it contains no OS-specific code, only the gesture logic and the common API.

Interface II — OS-Specific Input Emulation

This layer converts a recognised gesture into something an operating system actually understands as input — keyboard keys, mouse movement, or clicks. Because input injection is implemented differently on each OS, this is the one place where the code diverges:

ing System	Mechanism	What It Does
	uinput	Creates virtual input devices in the kernel, so gestures appear as real keyboard/mouse events to any app
ws	SendInput()	The Win32 API call that injects synthetic keyboard/mouse input into the system

This two-layer split is the heart of the design: if a new OS needs supporting, only Interface II changes; if the gesture vocabulary grows, only Interface I changes.

4. Application Layer — What Actually Uses the Gestures

The top of the stack has two kinds of consumer, depending on what they want.

Generic Applications

Any standard program — a game, a media player, a slide-deck presenter — that already knows how to use a keyboard and mouse. Because Interface II injects gestures as ordinary input-device events, these applications need zero modification: the glove simply appears to them as a keyboard/mouse.

Custom Software

Software written specifically for the glove can bypass the input-emulation layer entirely and consume gesture data directly from Interface I. This gives access to richer information (raw sensor values, continuous hand pose) rather than the simplified on/off of a keystroke — useful for applications that need fine-grained control.

5. End-to-End Data Flow — One Gesture, Step by Step

Tracing a single hand movement through the whole system makes the layering concrete:

Step	Component	What Happens
1	Flex sensors + IMU	Fingers bend and the hand moves; resistance and IMU readings change
2	8:1 MUX	ESP-32 cycles the select pins, connecting each flex sensor to the ADC in turn
3	ESP-32 ADC + I2C	Reads each flex value from the MUX and the IMU over I2C; assembles a raw data stream
4	ESP-32 WiFi/Bluetooth	Transmits the raw stream wirelessly to the PC
5	Interface I	Receives the stream, processes it, and recognises the gesture
6	Interface II	Maps the gesture to a keyboard/mouse event (uinput on Linux, SendInput on Windows)
7	Application	The game, media player, or custom software receives the input and reacts

6. Why This Architecture

Every split in the design exists for a reason:

Decision	Why
Flex sensors + IMU together	Flex captures finger pose; IMU captures hand motion. Neither alone is enough for rich gestures.
MUX into a single ADC pin	Saves scarce analog pins on the ESP-32; the MUX time-shares one ADC across five sensors.
Send raw data, not gestures	Keeps the glove cheap and dumb; recognition can be upgraded on the PC with no firmware changes.
Split Interface I from Interface II	Isolates OS-specific input injection so the system is portable and easy to extend.
Two application paths	Standard apps get keyboard/mouse compatibility; custom apps get direct, richer access.

7. Component Summary

Component	Layer	One-Line Role
5 Flex Sensors	Sensor	Measure how much each finger is bent
9-Axis IMU	Sensor	Measure hand orientation and motion in space
8:1 Analog MUX	Acquisition	Time-share one ADC pin across five flex sensors
ESP-32	Acquisition	Read sensors, assemble the data, transmit wirelessly
Interface I	PC Software	Receive and process data; recognise gestures; expose a common API
Interface II	PC Software	Convert recognised gestures into OS-specific input events
uinput / SendInput	PC Software	The Linux and Windows mechanisms that inject input
Generic Applications	Application	Games, media, presentations using standard input
Custom Software	Application	Applications consuming gesture data directly